

Сопроводительная документация проекта «СтройКонтроль»

Задача: ЛЦТ 2026, кейс 07 — мониторинг строительной площадки по снимкам камер
Заказчик: Департамент градостроительной политики города Москвы
Ветка репозитория: `main`

Система берёт снимки с камер стройки, находит на них технику, сверяет её с календарным графиком и показывает отклонения D1–D10 со снимками-доказательствами.

1. Модель данных

У каждого сервиса своя база PostgreSQL 16. Между базами нет внешних ключей и прямых запросов: сервисы обмениваются данными только через REST API, ссылки — по UUID. Полное описание полей — `docs/data-model.md`.

1.1. Где что хранится

Хранилище	Владелец	Что внутри
<code>plandb</code>	<code>plan-service</code>	объекты (<code>object</code>), этапы графика (<code>stage</code>), правила «этап → техника» (<code>stage_rule</code>), справочник работ (<code>work_type</code>), рабочий календарь (<code>work_calendar</code>)
<code>sitedb</code>	<code>site-service</code>	камеры (<code>camera</code>), зоны на кадре (<code>zone</code>), снимки (<code>image</code>), детекции (<code>detection</code>), 30-минутные окна наблюдения (<code>session</code>), факты окна по участку и классу (<code>session_fact</code>), видимость участков (<code>area_visibility</code>), стадия объекта по снимку (<code>stage_observation</code>)
<code>analysisdb</code>	<code>analysis-service</code>	прогоны анализа (<code>analysis_run</code>), отклонения (<code>deviation</code>), настройки правил D1–D10 (<code>deviation_rule</code>), факт по этапам (<code>stage_fact</code>), активность по дням (<code>daily_activity</code>), загрузка техники по дням (<code>daily_equipment</code>), статус объекта (<code>object_status</code>)
S3 (SeaweedFS, сервис <code>s3</code>)	<code>site-service</code> , <code>analysis-service</code>	снимки (бакет <code>images</code>), PDF-отчёты (бакет <code>reports</code>). Отчёты не хранятся в базе: список берётся из бакета

Классы техники в базах не хранятся — только их коды. Единственный источник классов — файл `packages/contracts/equipment_classes.yaml`.

1.2. Техника

- Классы** — `packages/contracts/equipment_classes.yaml` : 16 классов (15 видов техники и `person`). У класса есть код, русское название, группа, английские промпты для детектора, признаки `transient` (приезжает и уезжает рейсами) и `works_in_place` (работает, не двигаясь: башенный кран, автокран, бетононасос) и метки внешних датасетов (`aliases`).
- Детекция** (`sitedb.detection`) — одна машина на одном снимке:
 - `image_id` , `session_id` , `camera_id` — снимок, окно, камера;
 - `equipment_class` — код класса;
 - `bbox` — рамка [`x1` , `y1` , `x2` , `y2`] , доли кадра от 0 до 1;
 - `conf` — уверенность модели;
 - `anchor` — середина нижней стороны рамки (точка контакта с землёй), по ней машина относится к зоне;
 - `zone_id` — зона, в которую попала точка; `null` — вне зон;
 - `moved` , `displacement` — сдвинулась ли машина с прошлого окна той же камеры;
 - `model_version` — какая модель дала детекцию.
- Факт окна** (`sitedb.session_fact`) — сколько машин класса было на участке за 30 минут (максимум по камерам, а не сумма), сколько из них стояли на месте, и снимки-доказательства.
- Статус машины** — `WORKING` , `IDLE` , `OUT_OF_ZONE` , `UNKNOWN` — считает `analysis-service` по фактам окон при каждом прогоне (`src/core/equipment_state.py`). Отдельной таблицы статусов нет. Итог по дням — в `analysisdb.daily_equipment`.

1.3. Этапы работ

Типы объектов (`packages/contracts/enums.yaml`): `RESIDENTIAL_MONOLITH` (монолитный жилой дом), `RESIDENTIAL_PANEL` (панельный жилой дом), `PUBLIC_BUILDING` (общественное здание), `ROAD` (дорога).

- Этап графика** (`plandb.stage`): код и название, фаза, плановые даты `plan_start` и `plan_end` , нормативная длительность в рабочих днях, связи с предшественниками (`predecessors` : тип связи и лаг), признак критического пути (`is_critical`), резерв, тип участка (`zone_type`), ожидаемая стадия на фото (`visual_stage`), источник (`GENERATED` / `IMPORT` / `MANUAL`) и отметка оператора «этап выполнен» (`completed_on`).
- Правило «этап → техника»** (`plandb.stage_rule`):
 - `required` — группы обязательной техники: «любой из классов, не меньше N единиц»;
 - `allowed` — допустимая техника, из-за неё D3 не возникает;

- `signature` — признак фактического старта: набор машин и/или стадия на фото;
 - `min_sessions` — сколько рабочих сессий подряд должно держаться условие (по умолчанию 2).
3. **Справочник работ** (`plandb.work_type`) — четырёхуровневый справочник строительных работ Москвы с отметками применимости по типам объектов.

Откуда берётся график:

- **Автоматический расчёт по MPP-3.2.81-12** (`POST /api/v1/plan/objects/{id}/plan/generate`) работает **только для монолитного жилого дома**. Нормы и шаблон этапов есть только для него (`services/plan-service/data/mrr_norms.json` , `wbs_templates.json`).
- **Импорт из CSV или XLSX** (`POST /api/v1/plan/objects/{id}/plan/import`) — для любого типа объекта. Подробнее — раздел 3.5.

1.4. Отклонения

Таблица `analysisdb.deviation`: - `object_id`, `stage_id`, `area` (участок), `equipment_class`, `session_id` — к чему относится; - `code` — D1–D10; - `severity` — INFO, LOW, MEDIUM, HIGH; у части правил растёт, если нарушение держится несколько рабочих дней; - `status` — NEW, CONFIRMED, REJECTED, RESOLVED (RESOLVED — условие перестало выполняться, отклонение закрыто автоматически); - `verdict`, `verdict_comment`, `verdict_by`, `verdict_at` — решение оператора (CONFIRMED / REJECTED), кто и почему; - `title`, `message` — текст карточки; - `facts` — все числа, из которых сделан вывод; - `evidence` — снимки и детекции-доказательства; - `rule_ref` — какое правило и какой версии работало; - `first_seen_at`, `last_seen_at`, `occurrences` — когда замечено и сколько сессий подряд.

Повторный прогон обновляет открытое отклонение, а не создаёт дубль.

Пороги и тексты правил — таблица `analysisdb.deviation_rule`. При первом старте она заполняется из `services/analysis-service/data/deviation_rules.yaml`, дальше правится через API и интерфейс.

1.5. Правила D1–D10

Правила считает `analysis-service`. «Сессия» — 30-минутное окно наблюдения в рабочие часы.

Код	Что значит	Когда срабатывает	Серьёзность
D1	Нет обязательной техники	На участке активного этапа нет ни одной машины из обязательных групп заданное число сессий подряд	MEDIUM, через 1 рабочий день — HIGH
D2	Неполный комплект	Часть обязательной техники есть, но не все группы правила выполнены	MEDIUM, через 3 рабочих дня — HIGH
D3	Техника не по этапу	2 сессии подряд на участке техника, которой нет в правилах активных этапов. Если она нужна будущему этапу участка — «возможное опережение»	MEDIUM
D4	Простой	Машина стоит без движения 3 сессии подряд. Транзитная техника и машины <code>works_in_place</code> на участке активного этапа простоем не считаются	LOW, через 1 рабочий день — MEDIUM
D5	Техника не в той зоне	2 сессии подряд нетранзитная машина на рабочем участке, где по плану нет активного этапа этого типа	MEDIUM
D6	Опасная зона	В размеченной опасной зоне замечена техника или человек (<code>person</code>)	HIGH
D7	Стадия по фото не совпадает с планом	2 сессии подряд стадия на фото отстаёт от плановой (или опережает её)	HIGH
D8	Этап затянулся	Плановый срок этапа прошёл, а признак его работ на участке всё ещё есть	HIGH
D9	Этап не начат в срок	За первые 3 рабочих дня этапа признак старта не появился, хотя участок был виден не меньше чем в половине сессий	HIGH
D10	Вне контроля ИИ	2 сессии подряд участок не виден: нет снимков, темно, кадры размыты, обзор перекрыт; или у активного этапа нет размеченной зоны. Выводы по участку не делаются, нужна ручная проверка	INFO

D1, D2, D8 и D9 работают только для этапов, у которых есть правило «этап → техника».

2. Описание кода

2.1. Состав репозитория

```
ConstructionSupervision/
├── apps/web/           # интерфейс: React 18, TypeScript, Vite, Tailwind CSS, TanStack Query
├── services/
│   ├── gateway/       # Nginx: единый вход на порту 8080, интерфейс, API, Swagger, снимки
│   ├── plan-service/  # план: объекты, график (импорт и генератор по MPP), правила этапов
│   ├── site-service/  # площадка: камеры, зоны, приём снимков, окна наблюдения;
│   │               # site-worker (arq + Redis) отправляет снимки на распознавание
│   ├── vision-service/# распознавание: детектор техники, стадия объекта, качество кадра
│   └── analysis-service/# анализ: правила D1-D10, прогресс и прогноз этапов,
│                       # модуль PDF-отчётов с резюме от LLM или по шаблону
├── packages/
│   ├── contracts/     # общие контракты: equipment_classes.yaml, enums.yaml, OpenAPI
│   ├── py-common/     # общий Python-код сервисов
│   └── ts-api-client/  # TypeScript-клиент API для интерфейса
├── ml/                # подготовка датасетов, дообучение детектора, оценка качества
├── data/              # демо-данные (data/seed) и веса моделей (data/models, не в git)
└── scripts/           # fetch_models.py, seed.py, e2e.py, health.py и др.
```

Генератор графика — модуль `plan-service`, отчёты — модуль `analysis-service`, отдельных сервисов у них нет. Кроме сервисов в `docker-compose.yml` есть PostgreSQL, SeaweedFS (`s3`), Redis, `llm` (`llama.cpp` с Gemma, профиль `llm`) и `tools` (контейнер для скриптов).

2.2. Устройство сервиса

Python-сервисы (FastAPI, Pydantic v2, SQLAlchemy 2.0) разложены по слоям: - `src/core/` — логика без базы и сети: правила D1–D10, критический путь, генератор графика, разбор импорта, метрики качества кадра; - `src/dal/` — модели таблиц и репозитории; - `src/api/` — HTTP-роутеры и схемы; - `src/services/` — сценарии: связывают ядро, базу и другие сервисы.

Контракты между сервисами — `packages/contracts/interservice.md`, архитектура — `docs/architecture.md`.

2.3. Комментарии

Каждый модуль `services/*/src/core/` начинается с докстринга: что он делает и на какой пункт ТЗ, методики (`docs/methodology.md`) или норматива опирается. Комментарии в коде объясняют, почему сделано так, а не пересказывают код. Пороги с пометкой «экспертное допущение» — наш выбор, а не норматив.

3. Условия и ограничения

3.1. Модели

Задача	Модель	Файл в <code>data/models/</code>
Детекция техники и людей	YOLO-World v2 small, дообученная <code>ce-ulima-v1</code>	<code>yolov8s-worldv2-ce-ulima-v1.pt</code> (задаётся в <code>VISION_DET_WEIGHTS</code>)
Кодирование промптов классов для YOLO-World	OpenAI CLIP ViT-B/32	<code>clip/ViT-B-32.pt</code>
Стадия объекта по снимку	OpenCLIP ViT-B-32 (<code>laion2b_s34b_b79k</code>), zero-shot по промптам из <code>services/vision-service/data/stage_prompts.yaml</code>	<code>openclip-vit-b32/open_clip_model.safetensors</code>
Резюме отчёта	Gemma 4 E4B, GGUF Q4_K_M, в контейнере <code>llm</code> (<code>llama.cpp</code>)	<code>llm/gemma-4-E4B-it-Q4_K_M.gguf</code>

- В сервисе работает один детектор — `ce-ulima-v1`. Это YOLO-World, дообученный на Construction Equipment и Лиме; классы по-прежнему задаются текстовыми промптами. `fetch_models.py` скачивает и исходные веса `yolov8s-worldev2.pt` (zero-shot) — их можно указать в `VISION_DET_WEIGHTS` вместо дообученных.
- Стадии: котлован (`PIT`), сваи (`PILES`), фундамент (`FOUNDATION`), каркас (`FRAME`), фасад (`FACADE`), благоустройство (`LANDSCAPING`).
- Качество кадра (`services/vision-service/src/core/quality.py`) — без нейросети: яркость и размытость (дисперсия лапласиана). Годен ли кадр, решает `site-service` по порогам `MIN_BRIGHTNESS` и `MAX_BLUR`. Непригодные кадры не участвуют в выводах, участок получает D10.
- LLM получает только готовые факты (этапы, даты, числа, коды отклонений), без снимков. Если модель выключена или не ответила вовремя, резюме строится по шаблону. Gemma можно заменить любой моделью с OpenAI-совместимым API (`LLM_BASE_URL` , `LLM_MODEL`) или отключить (`LLM_ENABLED=false`).

3.2. Данные для обучения и проверки

Набор	Что это	Как использован
Construction Equipment (Kaggle) , CCO	Около 20 000 кадров (16 754 train, 3 228 val) с неподвижных камер, 2023–2024, есть зима и ночь, 17 классов	Обучение детектора. Метки сведены к нашим классам через <code>aliases</code>
Universidad de Lima , CC BY 4.0	1 046 кадров 3840×2160 с 4 неподвижных камер одной стройки (ноябрь 2020 — февраль 2021), 8 классов	Разбит по сериям съёмки: 720 кадров — обучение, 62 — валидация, 264 — отложенный тест. Из него же демо-хронология
<code>1ct-test</code> — снимки организаторов	100 фото около десятка московских строек за 2023–2026 годы, лето и зима, 600 размеченных машин. Людей не размечали	Только проверка, в обучении не участвовал

Подробнее — `ml/README.md`, все числа — `docs/metrics.md`.

3.3. Качество детекции

Лима, тест (264 кадра, одна стройка): mAP50 zero-shot — 0,07, `ce-ulima-v1` — 0,81. Экскаватора в этом тесте нет: его качество здесь не измерено.

Снимки организаторов (100 снимков, 600 машин):

Показатель	Zero-shot yolov8s-worl dv2	<code>ce-ulima-v1</code> (в сервисе)
mAP50 (валидация Ultralytics)	0,16	0,11
mAP50, один класс на рамку, как в сервисе	0,07	0,13
Верно найдено при пороге 0,35	59 из 600	83 из 600
Ложных рамок при пороге 0,35	521	18

- Ни одна из моделей не годится для снимков организаторов как есть. `ce-ulima-v1` выбрана потому, что почти не выдумывает технику: находит каждую седьмую машину.
- Экскаваторы: 68 верных из 243 (у zero-shot — 1).
- Буровые и сваебойные установки не распознаются (0 из 143), автокраны почти не распознаются (1 из 89): таких машин нет в открытых наборах с подходящей лицензией. Из-за этого на реальных стройках возможны ложные D1 и D9 по этапам со сваями и кранами — такие отклонения оператор отклоняет вердиктом `REJECTED`.
- Качество отдельно ночью, зимой и по стадиям не измерено. Точность классификатора стадии не измерена: размеченной выборки нет.

Скорость (RTX 3060 Laptop; медиана / p90): - только детектор: 29 / 58 мс на снимках организаторов, 23 / 53 мс на кадрах 4К; - детектор и классификатор стадии: 56 / 76 мс на GPU (вход 1280), 900 / 985 мс на CPU (вход 960). Требования T3 — ≤ 100 мс на GPU и ≤ 1–2 с на CPU.

3.4. Классы техники

16 классов из `packages/contracts/equipment_classes.yaml`.

- 8 типов из T3:** самосвал (`dump_truck`), экскаватор (`excavator`), каток (`roller`), кран-манипулятор (`manipulator_crane`), автобетоносмеситель (`concrete_mixer`), бульдозер (`bulldozer`), грузовик (`truck`), автокран (`truck_crane`).
- Добавлены нами:** башенный кран (`tower_crane`), автобетононасос (`concrete_pump`), буровая/сваебойная установка (`pile_driver`), погрузчик (`loader`), асфальтоукладчик (`asphalt_paver`), грейдер (`grader`), малотоннажный грузовик (`light_truck`), человек (`person` — для опасных зон, D6).

Класс в списке не значит, что модель его хорошо узнаёт: см. раздел 3.3. Каток, бульдозер, асфальтоукладчик, грейдер и бетононасос почти не встречаются в тесте организаторов, их качество там не измерено.

3.5. Для каких объектов и этапов настроена логика

- Монолитный жилой дом (`RESIDENTIAL_MONOLITH`):** настроен полностью: нормы MPP-3.2.81-12, шаблон из 12 этапов с правилами «этап → техника» по таблице 5.2 T3 (`services/plan-service/data/wbs_templates.json`). Для него работает автоматический расчёт графика.
- Панельный дом, общественное здание, дорога:** шаблонов этапов, норм и правил для них нет. График загружается только из файла, правила «этап → техника» задаёт оператор в интерфейсе или через API. Пока правила нет, по этапу не работают D1, D2, D8, D9.

Импорт графика — `POST /api/v1/plan/objects/{id}/plan/import` или в интерфейсе «Построить график → Из файла CSV / XLSX»: - обязательные колонки: код, наименование, начало, окончание; - необязательные: фаза, тип участка, визуальная стадия, связи (например, 10.4–10.8 FS); - если код этапа есть в шаблоне монолитного дома, недостающие поля и правило берутся из шаблона. Иначе фаза и тип участка обязательны, а правило этапа нужно задать вручную; - примеры: `data/seed/demo_schedule.csv` (короткий, со связями) и `data/seed/schedule.xlsx` (график демо-объекта).

Этапы шаблона монолитного дома:

Код	Этап	Обязательная техника	Допустимая	Признак старта	Участок / стадия на фото
10.4-10.8	Подготовка территории	экскаватор ≥ 1 , самосвал ≥ 1	бульдозер, погрузчик, автокран	экскаватор + самосвал	пятно застройки / —
10.11	Обустройство площадки	манипулятор или грузовик ≥ 1	автокран, самосвал, каток	манипулятор	периметр / —
12.3.2	Ограждение котлована, сваи	буровая ≥ 1	автокран, миксер, бетононасос, экскаватор	буровая	котлован / сваи
12.3.1	Разработка котлована	экскаватор ≥ 1 , самосвал ≥ 2	бульдозер, погрузчик	экскаватор + самосвал	котлован / котлован
12.3.3	Основание	самосвал ≥ 1 , каток или бульдозер ≥ 1	погрузчик, миксер	каток	котлован / котлован
монтаж-крана	Монтаж башенного крана	автокран ≥ 1	грузовик	автокран + башенный кран	пятно застройки / —
12.3.4	Фундаментная плита, монолит ниже 0	миксер ≥ 1 , бетононасос ≥ 1	башенный кран, автокран, грузовик	миксер + бетононасос	котлован / фундамент
12.3.10	Гидроизоляция, обратная засыпка	экскаватор или погрузчик ≥ 1 , самосвал ≥ 1	каток	самосвал и стадия «фундамент»	котлован / фундамент
12.4.4	Надземный каркас	башенный кран ≥ 1	миксер, бетононасос, грузовик	башенный кран и стадия «каркас»	пятно застройки / каркас
12.4.8	Стены, кровля, фасад	башенный кран, автокран или манипулятор ≥ 1	грузовик	стадия «фасад»	пятно застройки / фасад
12.5.1	Наружные сети	экскаватор ≥ 1	самосвал, манипулятор, каток	экскаватор	периметр / —
12.7	Благоустройство	самосвал ≥ 1 , каток ≥ 1	асфальтоукладчик, погрузчик, бульдозер	каток	периметр / благоустройство

Правила — наше прочтение таблицы 5.2 ТЗ; тип участка, стадия и связи — экспертное допущение. Автовышки из ТЗ в шаблоне нет: такого класса нет. Этап 12.4.10 «Бетонирование колонн и перекрытий» есть только в демо-графике, его правило (миксер + бетононасос) ставит `seed.py` из `data/seed/rules.json`. Любое правило можно изменить в интерфейсе без перезапуска.

3.6. Прочие ограничения

- Камеры неподвижны: зоны размечаются на кадре каждой камеры.
- Ночью, в туман и при грязном объективе кадры отсекаются по качеству, участок получает D10.
- Лица и номера машин система не распознаёт и не хранит.
- Выводы о работе техники делаются по снимкам раз в 30 минут, а не по видео.

3.7. Лицензии

- ПО стека:** PostgreSQL (PostgreSQL License), Redis 7 (BSD-3-Clause), Nginx (BSD-2-Clause), SeaweedFS (Apache 2.0), FastAPI, Pydantic, SQLAlchemy, asynpg, arg, React, Vite, Tailwind CSS, TanStack Query, llama.cpp — MIT или Apache 2.0. SeaweedFS заменил MinIO: MinIO перестал выпускать готовые образы ([ADR-0016](#)).
- Модели:** OpenCLIP и OpenAI CLIP — MIT. Gemma 4 — условия Google для моделей Gemma; модель необязательна (раздел 3.1). Ultralytics YOLO-World — AGPL-3.0; для закрытого коммерческого использования нужна коммерческая лицензия Ultralytics. Детектор изолирован в `vision-service` и доступен другим сервисам только по REST API.
- Датасеты:** Construction Equipment — CC0, Universidad de Lima — CC BY 4.0 (указание авторства). Наборы с запретом коммерческого использования (MOCS, ACID и др.) не использовались (`ml/README.md`).

4. Установка и запуск

Всё запускается в Docker Compose. Python на машине не нужен: скрипты идут в контейнере `tools`.

4.1. Что нужно

- Linux с Docker Engine и плагином `compose` или Windows 10/11 с Docker Desktop (WSL2);
- от 16 ГБ памяти, около 35 ГБ на диске (образы около 22 ГБ, модели и кадры около 7 ГБ);
- видеокарта NVIDIA — не обязательна, но без неё распознавание и резюме в разы медленнее. На Linux нужны драйвер и NVIDIA Container Toolkit, на Windows — свежий драйвер.

4.2. Запуск по шагам

Шаг 1. Код и настройки

```
git clone <URL_РЕПОЗИТОРИЯ>
cd ConstructionSupervision
cp .env.example .env
```

В Windows PowerShell: `chcp 65001` (UTF-8 в консоли), затем `copy .env.example .env`.

Если есть видеокарта NVIDIA, добавьте в `.env`: - Linux: `echo "COMPOSE_FILE=docker-compose.yml:docker-compose.gpu.yml" >> .env` - Windows: `Add-Content .env "COMPOSE_FILE=docker-compose.yml;docker-compose.gpu.yml"`

Шаг 2. Модели и демо-кадры

```
docker compose run --rm tools python scripts/fetch_models.py
```

Скрипт качает в `data/models/` и `data/seed/` то, чего нет в git, и сверяет sha256. Оборванная загрузка не оставляет битых файлов.

Группы: - `models` — `yolov8s-worldv2.pt`, OpenCLIP ViT-B-32, OpenAI CLIP ViT-B/32; - `demo` — рабочий детектор `yolov8s-worldv2-ce-ulima-v1.pt` и 56 демо-кадров Лимы; - `llm` — Gemma 4 E4B, около 5 ГБ.

Флаги: `--skip llm` — без Gemma (если LLM выключена или своя); `--skip demo` — без демо-кадров и без рабочего детектора, тогда в `.env` укажите `VISION_DET_WEIGHTS=/models/yolov8s-worldv2.pt`; `--force` — скачать заново.

Шаг 3. Запуск

```
docker compose pull
docker compose up -d --wait
```

`--wait` ждёт, пока все сервисы ответят готовностью (2–4 минуты). Сборка из исходников вместо готовых образов — `docker compose up -d --build`. Без Gemma — `COMPOSE_PROFILES=` в `.env` (подробнее — `README.md`, «Своя LLM вместо Gemma»).

Шаг 4. Проверка готовности (по желанию)

```
docker compose run --rm tools python scripts/health.py
```

Код выхода 0 — все сервисы готовы.

Шаг 5. Демо-данные

```
docker compose run --rm tools python scripts/seed.py
```

Создаёт демо-объект, загружает график, правила, снимки четырёх дней наблюдения (19–22.10.2026), камеры и зоны, ждёт распознавания и запускает анализ.

Шаг 6. Сквозная проверка

```
docker compose run --rm tools python scripts/e2e.py
```

Проверяет на отдельном объекте из тех же данных: 1. найдены ровно 7 заложенных отклонений, каждое в свой день (`data/seed/expected.json`), и ничего лишнего; 2. у каждого отклонения, кроме D10, есть факты и снимки; 3. статус объекта на дашборде совпадает с ожидаемым; 4. после правки правила этапа 12.4.10 D2 исчезает, D1 остаётся; после отката лента снова совпадает со сценарием.

Код выхода 0 — всё сошлось.

Остановить — `docker compose down`, удалить ещё и данные — `docker compose down -v`. Порт 8080 занят — задайте другой `GATEWAY_PORT` в `.env` до запуска. Подробнее — `docs/runbook.md`.

4.3. Адреса

Всё идёт через gateway на порту 8080: - интерфейс — <http://localhost:8080/>; - Swagger по всем сервисам — <http://localhost:8080/docs> (сервис выбирается в списке «Select a definition»; Swagger UI грузится с CDN и без интернета не откроется); - OpenAPI сервисов — <http://localhost:8080/api/v1/{plan|site|analysis|vision}/openapi.json>; - API —

`http://localhost:8080/api/v1/{plan|site|analysis|vision}/...` ; - снимки и отчёты — `http://localhost:8080/storage/...` (по подписанным ссылкам из API); - готовность gateway — <http://localhost:8080/health>; готовность сервисов проверяет `scripts/health.py` ; - веб-интерфейс SeaweedFS — <http://localhost:23646> (ключи `S3_ACCESS_KEY` / `S3_SECRET_KEY` из `.env`).